StudyPlannerApp

AI-Powered Study Planner with Peer Matching

Sprint 1 Progress Report & Module Allocation

Team Members	Blessing Yabe (2021417034) Phil Samakayi
Supervisor	Mr. Martin Phiri
Department	Computing and Informatics, University of Zambia
Methodology	Unified Process (Craig Larman)
Tech Stack	React.js · Django REST Framework · MySQL
Date	16 September 2026

1. What Has Been Accomplished

Sprint 1 of StudyPlannerApp is in progress, following the Unified Process (UP) artifact sequence. The following analysis-phase deliverables have been completed and are ready for supervisor review:

Artifact	Summary
Vision Document (v1.0)	Defines the problem statement, stakeholders, product features and success criteria for StudyPlannerApp.
Use Case Model (v1.0)	Fully dressed use case specifications covering 15 use cases across two actors — Student and Administrator.
Domain Model Diagram (SVG)	A conceptual class diagram capturing 10 domain classes and their relationships.
Domain Model Document (v1.0, .docx)	Documents Larman stereotype/category mapping (Entity, Boundary, Control-style conceptual classes), the association table, and the domain glossary.

A key architectural decision was also finalised during this sprint: the AI peer-matching module will take **transcript upload with OCR parsing** as its input, rather than self-assessment forms. This supersedes an earlier self-assessment approach referenced in a supplementary UML document, and is now the confirmed design for the MVP.

2. What Remains To Be Done

Two analysis/design artifacts remain before Sprint 1 is considered complete and the team moves into environment setup and implementation:

- **System Sequence Diagrams (SSDs)** — one per primary use case, showing system events between actors and the system-as-black-box.
- **Operation Contracts** — formal pre-/post-condition specifications for each system operation identified in the SSDs.

Once these two artifacts are signed off, Sprint 1 is closed and the team proceeds to environment setup (React + Django REST + MySQL scaffolding) and the first construction iteration, including renaming the existing `assessments` Django app to `profiles/transcripts` to reflect the transcript-upload approach.

3. Module Breakdown & Assignment

Implementation work is divided into six modules, mapped to the existing Django app structure (`accounts`, `profiles` [renamed from `assessments`], `matching`, `scheduling`, `subjects`, `config`) plus the React frontend. Backend/AI-pipeline modules are assigned to Blessing; scheduling, catalog and frontend modules are assigned to Phil, reflecting a natural split between the transcript→matching data pipeline and the scheduling/UI side of the app.

3.1 Blessing Yabe — Backend & AI Pipeline

Module	Django App	Scope
Accounts & Authentication	accounts	Student/Administrator registration, login, role-based access control, session/token management.
Transcript & Profile Module	profiles (renamed from assessments)	Transcript upload endpoint, OCR parsing pipeline, extraction of subjects/grades into structured student profile data.
AI Matching Module	matching	Peer-matching algorithm consuming parsed transcript data to recommend GroupStudy partners with compatible subjects/performance profiles.

3.2 Phil Samakayi — Scheduling, Catalog & Frontend

Module	Django App	Scope
Scheduling Module	scheduling	GroupStudy session creation, availability management, calendar/reminder logic.
Subjects & Catalog Module	subjects	Subject/course catalog management and curriculum mapping used by profiles and matching.
Frontend (React.js)	n/a — React app	Page views, component library, API integration layer consuming the Django REST endpoints for all modules.

Shared: the `config` app (project settings, environment configuration, URL routing) and overall API contract/integration testing are owned jointly.

4. Design Concepts & Principles Applied

4.1 Process & Modelling Approach

- **Unified Process (Larman)** — use-case driven, architecture-centric, iterative and incremental development; Sprint 1 corresponds to the Inception/early Elaboration artifacts (Vision, Use Cases, Domain Model, SSDs, Operation Contracts).
- **Domain modelling with Larman's conceptual-class categories** — the 10 domain classes were classified using Larman's category checklist (e.g. transactions, roles, containers, descriptions) rather than jumping straight to a design-class diagram, keeping analysis free of implementation detail.
- **Use-case driven traceability** — each of the 15 use cases traces forward to an SSD and an Operation Contract, keeping requirements, behaviour and implementation aligned.

4.2 GRASP & Object-Design Principles (planned for Design phase)

- **Information Expert** — assigning responsibilities (e.g. computing a match score) to the class that holds the necessary data, such as the Student/Transcript profile.
- **Controller** — each Django app exposes REST views/viewsets acting as controllers that receive requests from the React frontend and delegate to domain/service logic.
- **High Cohesion / Low Coupling** — reflected directly in the modular app split (accounts, profiles, matching, scheduling, subjects), each owning a single responsibility area.
- **Separation of Concerns** — presentation (React), application/business logic (Django REST services), and persistence (MySQL) are kept in distinct layers.
- **Single Responsibility & Polymorphism** — anticipated during detailed design of the matching algorithm, where different matching strategies can be encapsulated behind a common interface.

5. System Architecture

StudyPlannerApp follows a three-tier, layered client-server architecture with a clear separation between presentation, application and data layers:

Layer	Technology	Responsibility
Presentation Layer	React.js	Single-page application; student and admin dashboards, transcript upload UI, GroupStudy scheduling views; communicates with the backend exclusively via REST API calls.
Application Layer	Django REST Framework	Exposes REST endpoints per app (accounts, profiles, matching, scheduling, subjects); hosts business logic, authentication, and the OCR/matching processing pipeline.
Data Layer	MySQL	Persists user accounts, parsed transcript/profile data, subject catalog, matching results and scheduling records.

5.1 Data Flow — AI Matching Pipeline

The core differentiating flow of the system is the transcript-driven matching pipeline: a student uploads a transcript (image/PDF) through React → the profiles app stores the file and triggers OCR parsing → extracted subjects/grades are normalised into structured profile data → the matching module consumes this data, together with the subjects catalog, to compute and rank compatible GroupStudy peers → results are served back to the frontend and used by the scheduling module to propose session times.

5.2 Architectural Style

- **Client-server / REST architecture** — stateless REST API between React client and Django server, enabling independent development and testing of frontend and backend.
- **Modular monolith** at the backend — one Django project, cleanly separated into feature apps (accounts, profiles, matching, scheduling, subjects, config), which keeps deployment simple for an MVP while preserving clear module boundaries for the two-person team.
- **Pipeline/pipe-and-filter pattern** within the profiles→matching flow — OCR extraction, normalisation and match-scoring are treated as discrete, sequential processing stages.